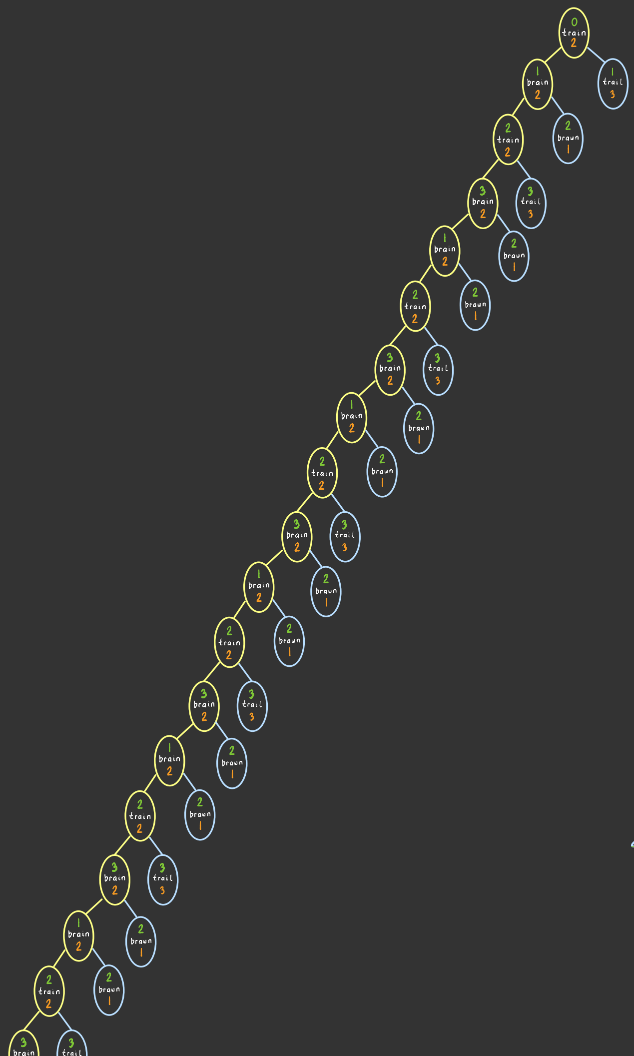
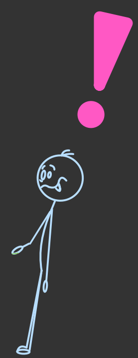
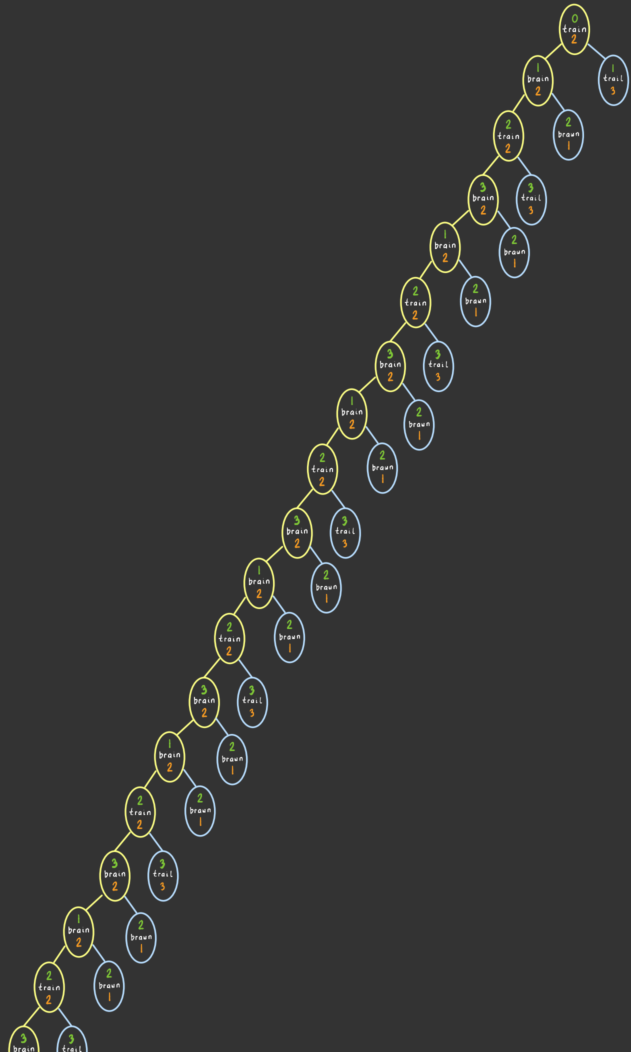
memoized search

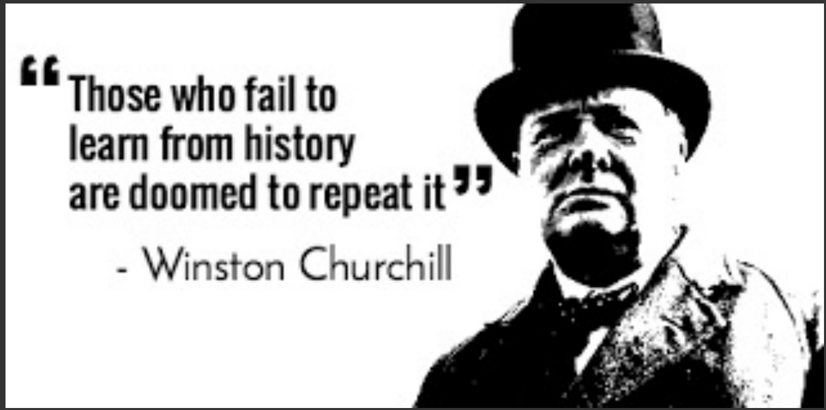
CSCI 270

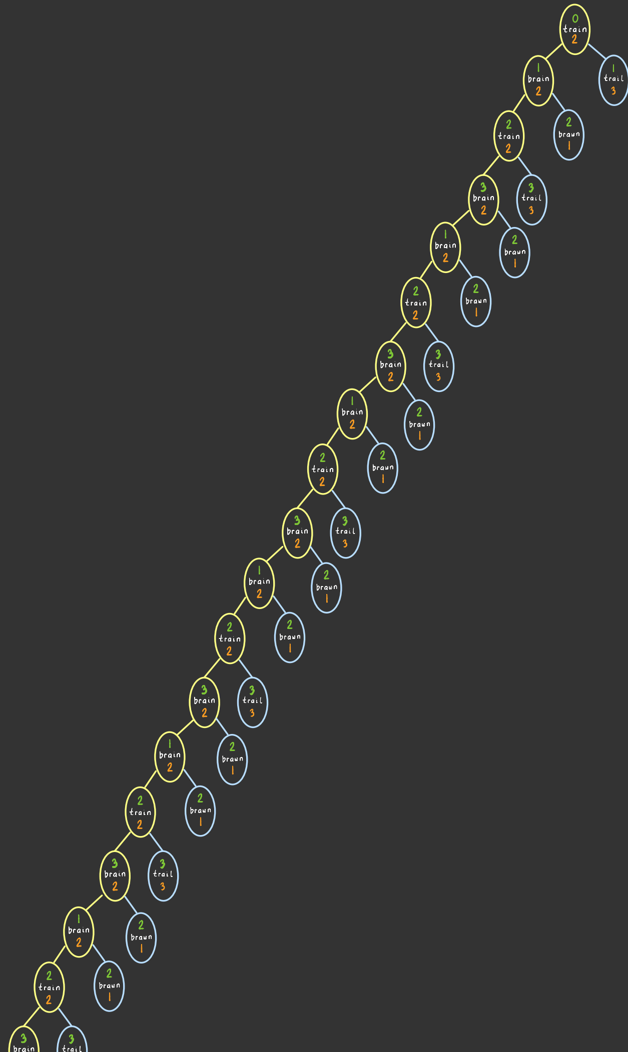


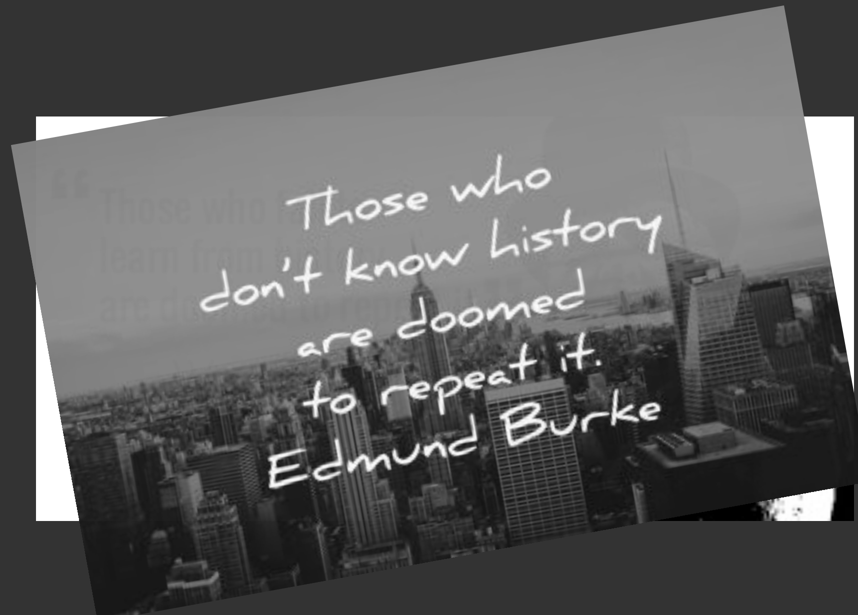
can't we avoid revisiting the same states over and over?

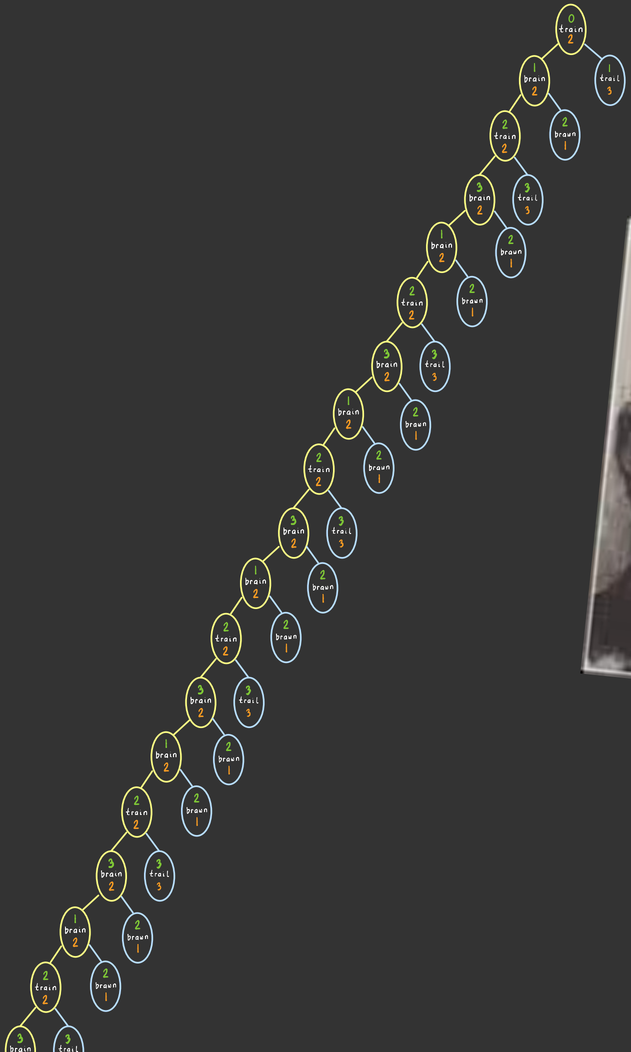


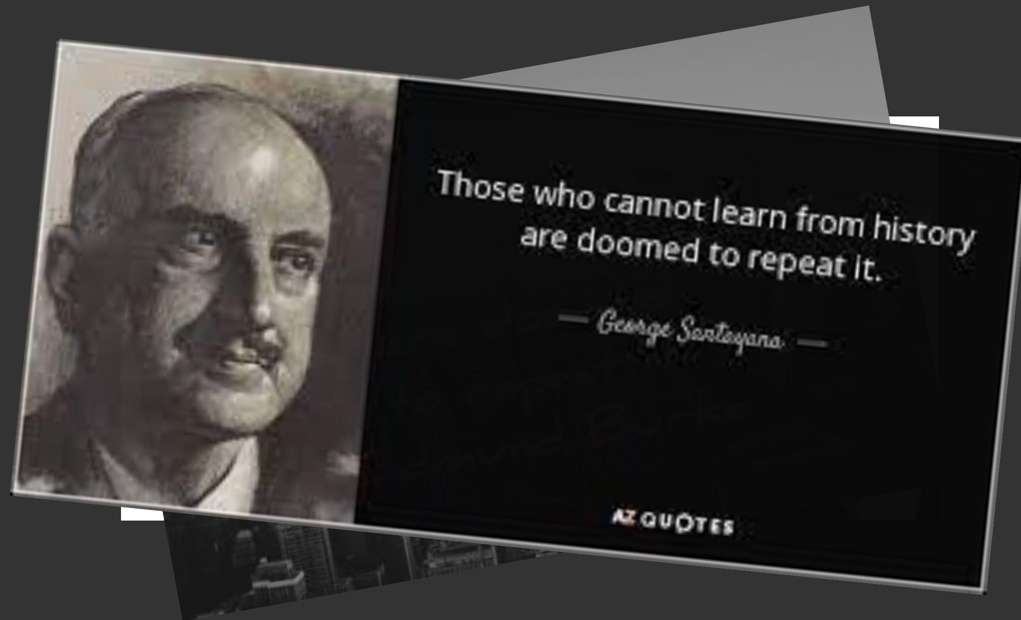

0 train 2
1 brain 2
1 trail 3
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain
3 trail


"Those who fail to
learn from history
are doomed to repeat it"
- Winston Churchill


0 train 2
1 brain 2
1 trail 3
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain
3 trail


Those who
don't know history
are doomed
to repeat it.
Edmund Burke


0 train 2
1 brain 2
1 trail 3
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain 2
3 trail 3
1 brain 2
2 brawn 1
2 train 2
2 brawn 1
3 brain
3 trail


Those who cannot learn from history are doomed to repeat it.
— George Santayana —
AZ QUOTES

can we adapt this so we don't revisit states?

search$(M = (Q, \Sigma, \Delta, q_0, F, \omega), H)$:

- container = new Container()
- container.put($\langle q_0, 0, H(q_0) \rangle$)
- repeat:
  - if container.empty() then return ∞
  - n = container.get()
  - if $q(n) \in F$ then return $g(n)$
  - for each transition $(q(n), \sigma, q') \in \Delta$:

    successor = $\langle q', g(n) + \omega(q(n), \sigma, q'), H(q') \rangle$

    container.put(successor)

"visit" a node

can we adapt this so we don't revisit states?

```
Search(M = (Q, Σ, Δ, q₀, F, ω), H):
▸ [ ??? ]
▸ container = new Container()
▸ container.put(⟨q₀, 0, H(q₀)⟩)
▸ repeat:
    ▸ if container.empty() then return ∞
    ▸ n = container.get()
    ▸ [ ??? ]
        ▸ [ ??? ]
        ▸ if q(n) ∈ F then return g(n)
        ▸ for each transition (q(n), σ, q') ∈ Δ:
              successor = ⟨q', g(n) + ω(q(n), σ, q'), H(q')⟩
              container.put(successor)
```

"visit" a node

yes.

but what are the performance implications?

```
memosearch(M = (Q, Σ, Δ, q₀, F, ω), H):
  ▸ visited = {}
  ▸ container = new Container()
  ▸ container.put(⟨q₀, 0, H(q₀)⟩)
  ▸ repeat:
      ▸ if container.empty() then return ∞
      ▸ n = container.get()
      ▸ if q(n) ∉ visited:
          ▸ visited = visited ∪ {q(n)}
          ▸ if q(n) ∈ F then return g(n)
          ▸ for each transition (q(n), σ, q′) ∈ Δ:
                successor = ⟨q′, g(n) + ω(q(n), σ, q′), H(q′)⟩
                container.put(successor)
```

"visit" a node

we trade

space

for

time

(to be continued)

memosearch$(M = (Q, \Sigma, \Delta, q_0, F, \omega), H)$:

- visited = { }
- container = new Container()
- container.put($\langle q_0, 0, H(q_0)\rangle$)
- repeat:
  - if container.empty() then return ∞
  - n = container.get()
  - if $q(n) \notin$ visited:
    - visited = visited $\cup \{q(n)\}$
    - if $q(n) \in F$ then return $g(n)$
    - for each transition $(q(n), \sigma, q') \in \Delta$:

      successor = $\langle q', g(n) + \omega(q(n), \sigma, q'), H(q')\rangle$

      container.put(successor)

"visit" a node